

Task 1. 数据类型与运算基础

1. Java 的八种基本数据类型分别是什么？

- 整数：byte、short、int、long
- 浮点类型：float、double
- 字符类型：char
- 布尔类型：boolean

2. 四种整型类型各占多少字节，表示范围大致是多少？

- byte：1 字节，范围 -128 ~ 127
- short：2 字节，范围 -32768 ~ 32767
- int：4 字节，范围约 -21亿 ~ 21亿
- long：8 字节，范围负2的63次方到2的63次方减1

3. 解释基本类型与引用类型的区别

	基本类型	引用类型
存储内容	具体的值	对象的地址
内存分配	在栈上分配	在堆上分配
默认值	每种基本类型有对应默认值	null
传递方式	复制一份，不影响原变量	传地址，影响原变量
比较类型	直接用 ==	== 比较地址，自定义 equals() 方法
包含范围	八种	几乎涵盖所有类型

4. 这里发生的是隐式类型转换还是显式类型转换？

- 隐式转换，显式转换会加 (类型)

5. b 的值是多少？请解释为什么

- c 是 char 类型，作为 int 类型运算时按 ASCII 码转成 48， $b = 48 + 4 = 52$

6. 写出运行结果，并解释每一步计算过程

- 结果

```
13
6 8
```

- 过程

```
int a = 5; // 将 a 赋值为 5
int b = 7; // 将 b 赋值为 7
int c = (++a) + (b++); // a 先自增再返回 6, b 先返回 7 再自增, c = 6 + 7 = 13
System.out.println(c); // 打印 13, 换行
System.out.println(a + " " + b); // 连接多个输出, 6空格8
```

7. 为什么“两个正数相加结果可能变成负数”? 请从二进制存储和溢出的角度解释

- 在补码的表示中, 最高位是符号位, 0 表示整数, 1 表示负数
- 做二进制加法时, 若和超出该整数类型的表示范围, 最高位可能因为进位变成 1, 导致正数变成负数

8. float 为什么可能出现精度问题?

- Java 的 float 占 32 位, 第 1 位符号位, 第 2 到 9 位阶码, 剩余 23 位尾数
- 符号位确定符号, 阶码控制小数点的左右移, 尾数代表实际数字
- 当有效数字超出可表示范围时, 超出部分被舍去, 造成精度损失
- 存储时小数部分只能表示 2 的负 n 次方, 类似 0.1 这样的小数只能存近似值, 造成精度损失

9.

```
Integer x = new Integer(18); // 创建一个 Integer 对象, value 为 18
Integer y = new Integer(18);
System.out.println(x == y); // 比较地址, false

Integer z = Integer.valueOf(18); // 调用 valueOf 方法, 返回缓存数组中的对象
Integer k = Integer.valueOf(18);
System.out.println(z == k); // true, 18 在缓存范围内

Integer m = Integer.valueOf(300);
Integer p = Integer.valueOf(300);
System.out.println(m == p); // false, 300 超出缓存
```

Task 2. 控制流与基础算法

2.1 if-else 与 switch

```
// 判断传入年份是否为闰年
boolean isLeapYear(int year) {
    if(year % 400 == 0) {
        return true;
    } else if(year % 100 == 0) {
        return false;
    } else if(year % 4 == 0) {
        return true;
    } else {
        return false;
    }
}
```

2.1.2 switch-case 和 if-else 的适用场景分别是什么？

- switch：单值或单表达式匹配，分支较多；
- if-else：多个变量参与判断，判断变量范围，分支较少

2.1.3 有人说 switch-case 是 if-else 的“语法糖”，你是否同意？请说出你的理由

- 不同意

switch	if-else
有贯穿行为	没有贯穿行为
只能比较有限类型	任意的布尔表达式
会生成跳转表，通过索引跳转匹配	只能按顺序匹配

2.2 循环

```
void print(int n) {
    int mid = n / 2;
    for (int i = 0; i < n; i++) {
        int offset = Math.abs(i - mid);
        int leadingSpaces = offset;
        int innerSpaces = n - 2 - 2 * offset;

        for (int s = 0; s < leadingSpaces; s++) System.out.print(" ");
        if (innerSpaces <= 0) {
            System.out.print("*");
        } else {
            System.out.print("*");
            for (int s = 0; s < innerSpaces; s++) System.out.print(" ");
            System.out.print("*");
        }
        System.out.println();
    }
}
```

2.3 递归与迭代

```
int fibonacci(int n) {
    if(n == 1 || n == 2) {
        return 1;
    }

    return fibonacci(n - 1) + fibonacci(n - 2);
}

int fibonacci(int n) {
    if(n == 1) {
```

```

        return 1;
    }

    int prev = 0, curr = 1;

    for(int i = 2; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}

```

2.3.1 递归和迭代的核心区别是什么？

- 递归：形式为函数调用自身，状态以栈的形式保存，结果从最底层返回，将大问题分解成小问题，结束条件为遇到基准情况
- 迭代：循环结构，维护循环变量，结束条件为条件不满足

2.3.2 为什么很多场景下更偏好迭代？

- 递归通常会占用更多内存，而迭代一般占用内存较少
- 递归深度过大可能造成栈溢出，迭代则不容易造成
- 递归会多次调用函数，开销较大
- 递归当深度过大时，会难以预测结果，但一般来说迭代的结果是更容易预测的

2.3.4 循环是否总能被递归取代？如果能，代价是什么？

- 理论上可以，但可能占用更大空间
- 栈溢出风险更大
- 降低了可读性

2.4 汉诺塔

```

public void hanoi(int n, char from, char aux, char to) {
    if (n == 1) {
        System.out.println("盘子 1 从 " + from + " 移动到 " + to);
        return;
    }
    hanoi(n - 1, from, to, aux);    // 把上面 n - 1 个盘子移到 aux
    System.out.println("盘子 " + n + " 从 " + from + " 移动到 " + to);
    hanoi(n - 1, aux, from, to);    // 把 n - 1 个盘子从 aux 移到 to
}

```

Task 3. 异常处理与程序健壮性

1. Error 和 Exception 的区别是什么？面对它们时，程序的处理态度应该有什么不同？

- Error 是程序无法处理的问题，由 JVM 或系统直接抛出，可被 Throwable 捕获，但不应该被捕获，编译期间检查不出来

- Exception 程序可以处理的问题，由程序逻辑问题或环境问题引起，可以被捕获，有些异常编译期间可以检查
- 面对 Error，直接抛出，中断程序，在日志、监控、告警中寻找问题
- 面对 Exception，用 try-catch 包裹或直接向上抛出，等待上层处理

2. checked exception 和 unchecked exception 的区别是什么？它们通常分别由什么原因引起？

- Checked Exception 编译期间强制检查，必须 try-catch 包裹或抛出
- Unchecked Exception 编译时不会检查，不强制要求处理
- 原因：Checked Exception 可能是外部环境、可预期的失败，Unchecked Exception 一般来自程序逻辑错误

```
public static void add() {
    try(BufferedReader br = new BufferedReader(
        new FileReader(new File("data.txt")))) {

        float sum = 0;
        int lines = 0;
        String line;
        while((line = br.readLine()) != null) {
            lines++;
            line = line.trim();
            sum += Integer.parseInt(line);
        }
        if(lines == 0) {
            throws EmptyFileException;
        }

        System.out.println(sum / lines);

    } catch (IOException | NumberFormatException e) {
        System.out.println("异常:" + e.getMessage());
    }
}
```

```
public class EmptyFileException extends Exception{

    private int code;
    private String msg;

    public EmptyFileException() {}

    public EmptyFileException(int code, String msg) {
        this.code = code;
        this.msg = msg;
    }

    public int getCode() {
        return code;
    }
}
```

```
}

public String getMsg() {
    return msg;
}
}
```

Task 4. Stream 流

1. 直接打印 limit, 会得到什么类型的结果?

- 得到 Object 的 toString() 方法生成的流对象

2. 为什么它不会直接把流里的元素打印出来?

- 没有使用终结方法

3.

```
List<String> passingStudents = students.stream()
    .filter(s -> s.getScore() >= 60)
    .map(Student::getName)
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
```